

Q1.

For a matrix of size 1000 x 1000 with seed = 5 averaging 10 runs:

Probability	Parallel (ms)	Sequential (ms)
0.05	6.641099	12.6505956
0.2	7.5477626	16.5146882
0.5	9.385954	24.6995506

There were several issues regarding performance that arose when I tested my initial implementation that led to this implementation. Initially, I set `omp_set_num_threads(n)`, which caused the performance of the parallelized execution to take much longer than that of the sequential execution. This is because when the value of `n` is large, the number of threads created would be much larger than the number of CPU cores available to be used for multithreading. Due to this, many threads are forced to wait for other threads to finish before they can act by utilizing the CPU core, drastically increasing the execution time of the program. By instead setting `omp_set_num_threads(omp_get_max_threads())`, this ensures that the number of threads utilized is equal to the number of CPU cores available, allowing all threads to act in parallel.

Additionally, another issue that arose was testing with insufficiently large `n` values. When the load of a thread is low, the overhead needed to create the threads and for thread synchronization exceeds the benefits of parallelism from multithreading. This led to the parallelized execution to take longer than the sequential execution.

As seen from the performance data, lower probability causes less execution time and better performance. This is due to lower probability matrices to be more sparse, leading to more 0's contained within the matrix. As more 0's leads to less writes to the `rowptr`, `cols`, and `vals` arrays, more cache misses occur as the cache is less frequently accessed, causing higher execution time and worse performance.

Q2.

For a random string of size 100,000:

Number of Optimistic Threads	Time Taken (Averaged over 10 runs)	Speedup
0	1.9446369	1
1	1.1918657	1.63159062
2	0.8816206	2.20575257
3	0.7528206	2.58313455
4	0.7021383	2.7695924

We can clearly see speedup when $t > 0$. As we are parallelizing the DFA transformation of the input string such that each thread takes a section of the input string and checks the possible transitions for each potential starting state based on the section of the string each thread handles and stores it in a 2d array (where each final state is stored corresponding to the section of the input string that each thread evaluates). The final calculation to determine the final state of the input string would thus check the 2d array, inputting the OK state as the initial state, and goes through every thread's potential outputs based on the previous state to determine the final state. As calculating the potential transitions is conducted in parallel, this would lower the amount of time it takes the program to conduct this task, reducing the overall time of the program, increasing performance, and causing more speedup. As we are testing on a very long string, the threads conduct more parallelized work than the cost of the overhead of the OpenMP threads, allowing for speedup when increasing the number of threads.